

Speaker Notes

Contents

- [1 1. Practicum in Operating Systems](#)
- [2 2. What we'll cover](#)
- [3 3. Takeaways](#)
- [4 4. Why this course exists](#)
- [5 5. Who runs this course](#)
- [6 6. You can define a system call. You have not seen one.](#)
- [7 7. Operating systems went back to theory in 1979](#)
- [8 8. Looking and reciting are different skills](#)
- [9 9. Thirty seconds, silently, to yourself](#)
- [10 10. What a practicum asks of you](#)
- [11 11. The course](#)
- [12 12. One idea carries all eight topics](#)
- [13 13. Topic 1 – I/O: files and the network](#)
- [14 14. Topic 2 – The system call boundary](#)
- [15 15. Topic 3 – Observing a running system](#)
- [16 16. Topic 4 – Virtual memory and the page cache](#)
- [17 17. Topic 5 – Protection, shared memory and IPC](#)
- [18 18. Topic 6 – Processes, threads and scheduling](#)
- [19 19. Topic 7 – Post-mortem analysis, with and without an agent](#)
- [20 20. Topic 8 – Leaks in the abstraction](#)
- [21 21. How you are assessed](#)
- [22 22. Two components: course credit and exam](#)
- [23 23. Three assignments, and the real dates](#)
- [24 24. The homework is a loop, not a test](#)
- [25 25. What passing an assignment buys you](#)
- [26 26. An AI grades you, and a human is on the loop](#)
- [27 27. How this arrangement fails](#)
- [28 28. What this course is betting](#)
- [29 29. Where to keep learning](#)
- [30 30. What must work on your Windows machine](#)
- [31 31. What must work on your Linux machine](#)
- [32 32. Two things that cost an evening](#)
- [33 33. Summary – what follows from this](#)

34 34. References

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

Slides: 34 · **Duration:** 28 min · **Venue:** Faculty of Informatics, Masaryk University (FI MUNI)

11. Practicum in Operating Systems

On screen:

PB158 · Lecture 1

Course orientation

Ondřej (Ondra) Krajíček me@ondrejkrasicek.com · [linkedin.com/in/OndrejKrajicek](https://www.linkedin.com/in/OndrejKrajicek) Built at render time

Good morning. This is PB158, Practicum in Operating Systems, and this is the first of two lectures today. This one runs about half an hour. It carries no operating-system content at all.

It tells you who teaches this course, what it covers and how it grades you. It also tells you what must work on your laptop before the first lab. Then a short break follows, and the second lecture starts the actual subject.

The next thirty minutes are the administrative half. I claim here that the administration is not separate from the subject. How a course measures you tells you what it thinks the subject is.

2 2. What we'll cover

On screen:

Lecture 1

This course grades what you can see a system do, not what you recite. [speculative]

1

Who and why

The gap a practicum fills beside your theory courses.

2

*The topics**Eight topic areas, and what each one lets you do.*

3

*Assessment**Two components, three assignments. A pass removes its exam subject. [speculative]*

4

*Reading**Four people who write well about this material.*

5

*Your machine**What to install before the first lab.*

That first line is the thesis of the whole term. Part one argues for it. Five parts, and the order matters. Part three is the one that decides your marks.

The headline comes now rather than later. Three homework assignments this semester. They are pass or fail, and there are no points. If your team passes an assignment, the course removes that assignment's subject from your exam. Pass all three and your exam keeps only an irreducible core. [speculative]

Part three gives the exact dates and the exact mechanism. Part one is four minutes on why this course exists, because everything in part three follows from it. If the assessment looks arbitrary, it is because you have not heard part one yet. [speculative]

3 3. Takeaways

On screen:

Lecture 1 · Hold these

1

*[T1] The course grades looking**Not reciting. The assessment in part 3 is built around that, and so is every topic.*

2

[T2] It works downward

From the code you write to the operating system under it. Each topic takes one thing you already use and shows the mechanism.

Two claims. If you remember nothing else from this half hour, remember these.

Two takeaways from this lecture, and both go up before either is argued for. First, this course grades you on what you can see a system do. It does not grade you on what you can recite about it. Everything in part three follows from that.

Second, the course works downward. It starts from code you already write, in Python and in C, and goes down through the layers to the operating system underneath. Every topic takes one abstraction you use daily, and shows you the mechanism it rests on.

Those are the two claims. Each is marked again on the slide that earns it, and both are repeated at the end.

4 4. Why this course exists

On screen:

Part I

The gap between what you can define and what you have seen.

Part one. Four minutes. Two things to establish here: who is responsible for this course, and what the course is for. The second one is the thesis of the whole term. Part one states that thesis directly. Part three then applies it to the rules.

5 5. Who runs this course

On screen:

Part I · Staff

Ondřej Krajiček

Lecturer. Write to krajicek@mail.muni.cz, or to the course contact in IS MUNI.

Co-lecturer, three assistants

Named in IS MUNI, with contact details, once the course opens. You meet the assistants in the labs.

*Material comes **from IS MUNI**. Work goes **to FI MUNI GitLab**, at `gitlab.fi.muni.cz`. Two systems, two directions. [speculative]*

Write early. Any question is reason enough, and email reaches the course staff between seminars.

Two lecturers teach this course together. Three teaching assistants join them in the labs. IS MUNI carries the roster, and this lecture leaves the names there. Until the course opens, the address on the slide is the route that works. The course contact in IS MUNI is the other route.

Two logistics. Material comes from IS MUNI. Slides, the handout and the assignment briefs all live there. Work goes the other way, to the faculty GitLab. Two systems, two directions.

Then two points about the course itself. It is mandatory, so some of you did not choose it. The aim here is to earn your attention rather than assume it. And write early. There are fewer than ten of you, so interrupt at any point. [speculative]

6 6. You can define a system call. You have not seen one.

On screen:

Part I · The gap

What you have

You can write Python. You can write C. You can define a process, a system call and virtual memory. [speculative]

What you have not

You never attached a tracer to a running program to read what it did. A tracer prints the calls a program makes. [speculative]

This is not a gap in you. It is a gap in the curriculum, and it dates from a licence change in 1979. [S5]

[S5] Lessons Learned from 30 Years of MINIX — Tanenbaum, CACM (2016). <https://www.cs.vu.nl/~ast/Publications/Papers/cacm-2016.pdf>

Here is the room, as this course describes it. You write Python and you write C. You have had operating systems as theory. You can define a system call, a process, virtual memory, a page. Nobody here attached a tracer to a running program and read what it printed. Nobody here used a debugger either.

That combination is strange. You can define the thing and you have never watched it happen. This is the normal output of a computer science curriculum. [speculative] The reason is historical. It goes back to a licence change in nineteen seventy nine. [S5]

7 7. Operating systems went back to theory in 1979

On screen:

Part I · History

In 1977 John Lions wrote a line-by-line commentary on the UNIX version 6 kernel. Hundreds of universities taught from it. [S5]

The 1979 AT&T release changed the licence and forbade teaching the source. Courses went back to theory-only mode or moved to toy simulators. [S5]

The teaching stayed theoretical [S5]. This course puts you in front of a tracer, attached to a running program. [speculative]

[S5] Lessons Learned from 30 Years of MINIX — Tanenbaum, CACM (2016). <https://www.cs.vu.nl/~ast/Publications/Papers/cacm-2016.pdf>

Andrew Tanenbaum tells this story in his retrospective on MINIX. John Lions wrote a commentary on the sixth edition UNIX kernel in nineteen seventy seven, and it became the textbook of hundreds of universities. Then the nineteen seventy nine release shipped with a licence that forbade writing about the source or teaching it.

Tanenbaum writes that operating systems courses went back to theory-only mode, or had to use toy simulators. He wrote MINIX to fix exactly that. Now, the interesting part is what happened next. The teaching stayed theoretical. [S5]

This course puts you in front of a tracer, attached to a running program. Topic three is where you meet the tools themselves, and what they cost. You have the theory already. What you do not have is the habit of looking. [speculative]

8 8. Looking and reciting are different skills

On screen:

Part I · Thesis

Reciting

You answer a question somebody already wrote down. A monitoring dashboard does this: "it answers questions you already knew to ask" [V2]

Looking

You ask a question nobody wrote down in advance, and the running system answers it. The field calls this observability: "the ability to investigate a system by asking any question about its behavior without needing to predict those questions in advance" [V1]

This course grades the second one [T1]. You can already recite more than you can predict. [speculative]

Here is the thesis of the course, in technical terms rather than as a slogan. The two words for this already exist. The first is monitoring. Monitoring answers questions somebody already knew to ask. Somebody chose the metric, set the threshold, wrote the alert.

A written theory test is monitoring. The second word is observability. The standard definition is the ability to investigate a system by asking any question about its behaviour, without needing to predict those questions in advance. That is what this course grades. The slide claims you can recite more than you can predict. [V1] [V2]

The next thirty seconds test that claim. Nobody has to answer aloud.

9 9. Thirty seconds, silently, to yourself

On screen:

Part I · Test it

In your head, say the steps between a Python `f.write()` returning and the bytes being on the disk.

Rozenblit and Keil measured what happens next. People rate their own understanding lower after they try to explain the thing than before. [S1]

If your explanation had a gap, that gap is the course. This course does not grade this test, and nobody has to say it aloud. [speculative]

[S1] The misunderstood limits of folk science: an illusion of explanatory depth — Rozenblit and Keil, Cognitive Science 26 (2002). cogdevlab.yale.edu

Take thirty seconds. Do not say anything aloud. In your own head, state the steps between a Python write call returning and your bytes reaching the disk. Name each layer. Wait thirty seconds in silence.

If you got two steps and stopped, that result has a name and a measurement. Rozenblit and Keil called it the illusion of explanatory depth. They asked people to rate their understanding, then to explain the thing, then to rate themselves again. The second rating was consistently lower. [S1]

The effect is strongest for explanations and weakest for plain facts. That is the shape of a theory-only education. Trying to explain reveals the gap. Hearing about it does not. [S1]

10 10. What a practicum asks of you

On screen:

Part I · Practicum

A lecture course asks you to reproduce an explanation. A practicum asks you to produce an observation and defend it. [speculative]

Crouch and Mazur measured the difference in a physics classroom. Students who only watched a demonstration gained little understanding of the concept it showed. Students who predicted the outcome first understood that concept measurably better. [S2]

So I will not prove the thesis in this lecture. The next lecture opens with a three-line program whose output disappears.

[S2] Classroom demonstrations: learning tools or entertainment? — Crouch, Fagen, Callan and Mazur, Am. J. Phys. 72 (2004). <https://works.swarthmore.edu/fac-physics/203/>

A practicum is not a lecture course with exercises attached. The difference is what it asks you to produce. A lecture course asks you to reproduce somebody else's explanation. A practicum asks you to produce an observation and then defend it.

Crouch, Fagen, Callan and Mazur measured this in a physics classroom. Passive watching produced very little understanding of the concept demonstrated. Predicting the outcome first produced a measurable gain in that understanding. Predict, then look. That is the method. [S2]

This lecture asserts the thesis of this course and proves nothing. That is deliberate. The next lecture opens with three lines of Python whose output vanishes. The proof arrives there.

11 11. The course

On screen:

Part II

Eight topic areas, and one idea that carries all of them.

Part two, and it is the longest part of this lecture. Twelve minutes. I name eight topic areas, and state what each one covers. One idea comes first, because the eight topics are eight instances of it rather than eight separate subjects.

12 12. One idea carries all eight topics

On screen:

Part II · The idea

Every operating-system interface keeps its promise only under conditions nobody wrote down. [inference]

The contract

The written half says what the call does. The scope clause says when it does that. [V4]

The method

You cannot deduce which layer departs from its documentation. Record the boundary crossings. Then count them. [V5]

A leak is your program leaving the scope, not the abstraction failing [V4]. A library's documentation is a claim. The system call it issues is a fact [V21]. Trust the trace when they disagree [inference].

[V4] [V5] *Where the Abstraction Leaks* · [V21] *AI Agents as Debugging Operators*

Here is the idea. Every abstraction the operating system gives you is a contract with a silent scope clause. The contract is written down. Open a file, write bytes, close it. You have read that part.

The scope clause is the part nobody writes down. It says under which conditions the contract holds. Step outside those conditions and you get a leak. The leak is not a bug. It is your program leaving the scope. [V4]

And here is why this course is built on tools. You cannot deduce which layer departs from its documentation. You have to make the boundary crossings visible and count them. [V5]

One sentence states the rule for the whole term. A library's documentation is a claim. The system call it issues is a fact. [V21] When the two disagree, trust the trace, and read the documentation as the wrong one. [inference]

13 13. Topic 1 — I/O: files and the network

On screen:

Part II · Topic 1

What it covers

One file write and one network request.

The general rule

Nearly every fast call returns before the work happens: "Almost every fast OS interface returns when the kernel has taken responsibility for the work, not when the work has happened." [V6]

The divergence

On Linux a socket is a file descriptor. On Windows it is not: "In Winsock applications, a socket descriptor is not a file descriptor and must be used with the Winsock functions." [V7]

The course starts here. Following one write down makes a tracer necessary [T2]. [speculative]

[V6] Where the Abstraction Leaks [V7] Sockets and Network I/O

Topic one is input and output, files and the network together. Everyone here has written a file, so the topic is already familiar. Here is the rule you meet. Almost every fast operating system interface returns when the kernel takes responsibility for the work. It does not return when the work happens. [V6]

Those are different moments. The gap between them is where the interesting failures live. Files and sockets share a slide because read and write operate on both. The platforms disagree sharply here. On Linux a socket is a file descriptor. On Windows, in Microsoft's own words, it is not. [V7]

Why start here? Following one write call down is what forces the boundary into view, and what makes a tracer necessary rather than decorative. The order rests on that argument alone. [speculative]

14 14. Topic 2 — The system call boundary

On screen:

Part II · Topic 2

What it covers

Where your program stops and the operating system begins.

The divergence

Linux publishes the system call interface. Windows publishes Win32 and reserves the layer below it. [V8]

The correction

Neither platform is less stable than the other. Each names a different layer as its contract: "Both are stable at the layer they nominate. The disagreement is about which layer is nominated" [V8]

You will be able to name where the boundary sits on each platform, and say why they chose differently. [speculative]

[V8] Windows and Linux place the stable syscall boundary at different layers — author's working notes, summarised in the handout appendix.

Topic two takes what topic one showed you in one case and makes it general. Where does your program stop and the operating system begin? Linux publishes the system call interface itself as the stable contract. You may issue a system call directly and the kernel promises it will keep working.

Windows does not do that. Windows publishes Win32 as the contract and treats the layer below as an implementation detail it reserves the right to change. Now, the easy mistake here is to read that as Linux doing it right.

Neither platform is less stable than the other. Each one names a different layer as the thing it promises to keep. The disagreement is about which layer gets nominated, and each choice buys something and costs something. The course works through both. [V8]

15 15. Topic 3 — Observing a running system

On screen:

Part II · Topic 3

What it covers

strace on Linux, NtTrace on Windows, and how to read what they print.

The honest limit

A tracer adds code, so it changes the timing it reports [inference]: "A tracer is not a window onto a running program. It is an edit to that program." [V22]

The precedent

A published graduate course "teaches kernel behaviour through DTrace and hardware performance counters" [V9] [S3]

You will be able to state a tracer's cost and its blind spot. [speculative]

[S3] Teaching OS with FreeBSD — Neville-Neil and colleagues (2017). https://www.usenix.org/system/files/login/articles/login_spring17_03_neville-neil.pdf · [V22]

[V25] Tracing cost and blind spot

Topic three is the tools. On Linux that is strace. On Windows it is NtTrace, which attaches at a different boundary for the reason given a moment ago. The limit matters most here. A tracer is not a window onto a running program. It is an edit to that program. [V22]

Three consequences follow:

- The tracer changes the timing it reports. [inference]
- The tracer costs real performance. [V22]
- Every tracing mechanism has a blind spot. [V25]

A tracer at the system call boundary sees only the work that crosses that boundary. Linux has an interface called `io_uring` that passes requests through shared memory instead, and `strace` never sees those. [V25]

You must leave this course able to run your platform's tracer, and able to state its blind spot. With only the first, the course has taught a command rather than a skill. [speculative] The design follows a published precedent. Neville-Neil and colleagues published a graduate course taught this way. [V9] [S3]

16 16. Topic 4 — Virtual memory and the page cache

On screen:

Part II · Topic 4

What it covers

Where the bytes you wrote went, and why freed memory is still held.

The mechanism

An allocation gives you address space, not memory: "A mapping can exist with no physical page behind it. The page is materialized on first touch, by a fault." [V17]

The consequence

One process yields several memory figures, each measuring a different thing. "How much memory does this process use" turns out to be a badly posed question [V17]

You will be able to read a memory figure and say which question it answers. [speculative]

[V17] Virtual Memory and the Page Cache — author's working notes, summarised in the handout appendix.

Topic four. You allocate memory and you think you now have memory. You do not. You have address space. A mapping can exist with no physical page behind it, and the page arrives on first touch, by a fault. [V17]

A page fault is not an error. It is the normal mechanism by which memory starts existing. Once you know that, several confusing things stop being confusing. Why a program that re-reads the same file is faster on its second run. Why your process monitor disagrees with your program.

Why free memory is mostly a bookkeeping category. The skill from this topic is reading a memory number and knowing which question it answers. [V17]

17 17. Topic 5 — Protection, shared memory and IPC

On screen:

Part II · Topic 5

What it covers

How one address space is isolated, and every way you deliberately cross that line.

The mechanism

The hardware checks every memory access before it completes: "Every load, store, and instruction fetch a process performs is checked before it completes." [V18].

Where it is checked

In the memory management unit, against the permission bits the kernel wrote. [V18]

You will be able to say why a segmentation fault is the protection working rather than failing. [speculative]

[V18] Memory Protection — author's working notes, summarised in the handout appendix.

Topic five. The hardware checks every memory access before it completes. Every load, every store and every instruction fetch your process performs is checked. The compiler plays no part in it. The language runtime plays no part in it. The kernel code plays no part in it. [V18]

The memory management unit does it, on every access, against the permission bits the kernel wrote. That is why isolation is cheap enough to be unconditional. It also re-frames something you all saw before. A segmentation fault is the protection working, and telling you so. [V18]

The second half of the topic covers the deliberate crossing. Shared memory, pipes, message queues and signals. Isolation is the default, and IPC is how you open one door in it. [speculative]

18 18. Topic 6 — Processes, threads and scheduling

On screen:

Part II · Topic 6

What it covers

What the operating system schedules, and how a runtime redefines your thread.

The finding

A runtime creates and switches its units inside the process, so nothing crosses the boundary: "A userspace-scheduled unit of concurrency produces no system call when it is created and no system call when it yields" [V19].

Why that matters

The trace of a concurrent program can be empty: "a syscall tracer shows nothing at all where the student expects to see concurrency" [V19]

Here the absence is the finding [inference]. You will read an empty trace as evidence. [speculative]

[V19] Processes, Threads and Scheduling — author's working notes.

Topic six contains a result that looks like a broken tool. You write a program using Python's async tasks, or goroutines, or green threads. You attach a tracer, expecting concurrency. You see nothing.

The runtime creates each unit inside your process, and switches between them inside your process. Both events stay on your side of the boundary, so the tracer has nothing to report. The trace looks like a serial program. [V19]

Students meet this and assume the tool is broken. It is not. The absence is the finding. It tells you the language runtime takes scheduling away from the kernel. Your thread is not what the operating system calls a task. Reading an absence as evidence is a genuinely hard skill, and this is where the course teaches it. [speculative]

19 19. Topic 7 — Post-mortem analysis, with and without an agent

On screen:

Part II · Topic 7

What it covers

Reading what a process left behind, and driving the tools with an AI agent.

Why it is separate

Every earlier topic needs a live process. Here it is dead: "A dump is all you get once it is already dead." [V20]

The agent question

An agent adds breadth over a large trace, and little on one stack frame [V26]. It reads only what the trace and the dump captured [V27]

You will be able to say which half of a session a model can take. [speculative]

[V20] Crash Dump Analysis · [V26] [V27] AI Agents as Debugging Operators

Topic seven is failure analysis, in two halves. The first is the crash dump. Every tool in topics one to six needs the process to be alive when you look. Post-mortem work begins where that stops being true. Tracing shows you a running system. A dump is all you get once it is dead. [V20]

It is a photograph, not a recording, and you reason backward from it. The second half is what an agent changes. That effect is tool-specific. [speculative] Reading a forty thousand line trace is breadth-shaped work, and a model is good at it. [V26]

Reasoning about one optimised-away local is depth-shaped, and a model is unremarkable. Knowing which is which is the skill. An agent reads the evidence a tool already produced. It acquires none of its own. [V27] [V21]

20 20. Topic 8 — Leaks in the abstraction

On screen:

Part II · Topic 8

The stance

A scope clause is the part of the contract nobody writes down [V4]. Topics 1 to 7 each end at one, and the last week puts the seven side by side. [speculative]

What you take away

Your own list of scope clauses a program stepped outside of. [speculative]

The pace

No fixed week per topic. Three weeks are reviews built from your submissions. [speculative]

By then your list has entries like a write that returned before the data was safe. [speculative]

[V4] Where the Abstraction Leaks — author's notes.

Topic eight is the synthesis lecture at the end of term. A scope clause is the part of the contract nobody writes down. [V4] Each of the seven topics before it ends at one, and the last week puts those cases side by side. You say what each one told you about the layer underneath. [speculative]

By then you will hold a personal list of scope clauses you watched a program walk out of. A write that returned before the data was safe. A memory figure measuring something other than memory.

One last thing, about pacing. There is no fixed week per topic. Three weeks are review lectures built from what your submissions got wrong, so nobody can state their content in September. You get the topics and the order. The pace follows the cohort. [speculative]

21 21. How you are assessed

On screen:

Part III

The rules, the dates, and the argument behind them.

Part three. Six minutes, and this is the part you will want to write down. The order is the rules, then the exact dates, then the reasoning. The reasoning belongs in an orientation rather than in a syllabus, and part one gave the argument. How a course measures you tells you what it thinks the subject is.

22 22. Two components: course credit and exam

On screen:

Part III · Components

Course credit (zápočet)

You earn it by submitting at least one of the three homework assignments. You do not have to pass one. The checker can fail your submission, and it still earns the course credit. [speculative]

Exam (zkouška)

No AI of any kind in the room, including any agent harness you bring. Passing an assignment shrinks it. [speculative]

Both rules above are specific to PB158, and you must pass both components. [speculative] I give the shrinking mechanism, and the reason for the AI rule, later in this part.

You earned a course credit before and you sat an exam before, so the two words keep their usual meaning. IS MUNI calls them zápočet and zkouška. Look for those words when you open your own record.

Two things are specific to PB158. First, the course credit. You earn it by submitting at least one of the three homework assignments. Submitting, not passing. The checker can fail your submission, and it still earns the course credit. You must pass the exam as well. [speculative]

Second, the exam. You write it in a classroom, and the room holds no AI. That rule covers any model or harness you bring. That rule sounds hard. The reason comes later in this part. The mechanism that shrinks the exam comes with it.

23 23. Three assignments, and the real dates

On screen:

Part III · Dates

1

set Tue 22 Sep

due Fri 2 Oct, midnight

reviewed Tue 6 Oct.

2

set Tue 6 Oct

due Fri 16 Oct, midnight

reviewed Tue 20 Oct.

3

set Tue 20 Oct

due Fri 30 Oct, midnight

reviewed Tue 3 Nov.

Each one asks you to observe what a system did, and to report what you concluded from it. The course grades that. [speculative]

Teams of two to three. You submit to FI MUNI GitLab at `gitlab.fi.muni.cz`. Pass or fail, and no points. [speculative]

Nothing is set today. The first assignment arrives next Tuesday, and you have ten days for it. [speculative]

Three assignments this semester. You work in teams of two to three. You submit to the faculty GitLab, at `gitlab dot fi dot muni dot cz`. Each one is pass or fail, with no points.

Each one asks you to observe what a system did, and to report what you concluded from it. The output is an observation you can defend, rather than a program that compiles. That is the whole of part one applied to your marks. [speculative]

Read the dates rather than compute them. Assignment one is set next Tuesday, the twenty second of September, and it is due at midnight on Friday the second of October. The review lecture covers it on Tuesday the sixth. Then the same shape twice more.

Set to due is ten days, not two weeks. Plan against two weeks and you lose a weekend. The deadline sits three days before the review, because the course staff need those days to read the submissions. Nothing is set today. [speculative]

24 24. The homework is a loop, not a test

On screen:

Part III · Shape

Resubmit as often as you like until the deadline. A fail costs time and nothing else. [speculative]

What the checker returns

Defects, not remedies. It names what is wrong, never the fix. [speculative]

Why not remedies

A critic is normally told to "suggest concrete revisions" [V15]. We drop that half on purpose. [speculative]

Push your failures. A passing commit is by construction the one with nothing left that the checker can catch. [speculative]

[V15] Generative Actor Critic — author's notes.

Now the shape of the homework, because it changes how you should work. You may re-submit as often as you like until the deadline. A fail before the deadline costs time and nothing else. So this is not a test with retries. It is a loop.

The loop has four steps. You submit. Something tells you what is wrong. You fix it. You submit again. [speculative]

One deliberate choice inside it. The checker returns defects, never remedies. It will say your program loses its output when the output is redirected. It will not say change line forty two. In the engineering version of this pattern the critic does both. [V15] This course drops the second half, because a checker that hands you the fix does the assignment for you.

And one consequence. A team that iterates to green has, by the deadline, removed every defect the checker tests for. So the teaching material is the sequence of attempts, not the pass. [speculative]

25 25. What passing an assignment buys you

On screen:

Part III · Reward

1

The exam has two parts

An irreducible core every student sits unaided, and a removable part. Its contents are not decided yet. [speculative]

2

A pass removes its subject

Passing an assignment removes that subject. You do not choose which subject. [speculative]

3

The whole team gets it

The submission is the team's, and every member's exam shrinks. [speculative]

A checker you may retry without limit becomes a target, so it stops being a good measure of you [V16] [inference]. Hence the unaided core. [speculative]

[V16] Goodhart's Law — author's notes.

This is the part that pays you. The exam has two parts. An irreducible core that every one of you sits, and a removable part. Passing an assignment removes that assignment's subject matter from your exam. Each assignment names its own subject, and you do not choose it. You know on the day it is set exactly what passing buys.

So it is a smaller syllabus, not merely a shorter paper. Pass all three and only the core is left. What is in the core is not decided yet, and you will get it in writing rather than discover it. [speculative]

The submission is the team's, so every member's exam shrinks. Yes, your exam depends on your teammates. Hear that on day one rather than in January. If your team cannot submit before a deadline, tell the course staff before it rather than after. Whether a student may submit alone is not decided. The answer comes in week two. [speculative]

Why keep a core? Goodhart's law. When a measure becomes a target it stops being a good measure, and an unlimited retry checker is a target. [V16] [inference] Goodhart names the problem and does not choose the remedy. The unaided core is this course's remedy. [speculative]

26 26. An AI grades you, and a human is on the loop

On screen:

Part III · The grader

An AI grades both the exam and the homework. A human stays on the loop. [speculative]

What that means

Nobody checks a grade before you get it. A human catches a wrong result afterwards: "the action does not wait for the human. It proceeds by default" [V10]

The asymmetry

You may not use AI in the exam, and an AI grades you. This lecture names it now rather than later. [speculative]

[V10] Human on the Loop — author's notes.

An AI grades both the exam and the homework. A human is always on the loop. The phrase on the loop is weaker than it sounds, so here is the precise meaning. The defining feature is negative. The action does not wait for the human. It proceeds by default, and the human's role is to catch it, not to authorise it. [V10]

Nobody checks before a fail is a fail. Then the asymmetry every student works out eventually. You may not use AI in the exam, and an AI grades you. This lecture names it now rather than leaving it to week six. They are different tasks. The exam measures what you can do unaided. [speculative]

27 27. How this arrangement fails

On screen:

Part III · Failure

Vigilance decays

Mackworth's watchers hunted rare double jumps in a pointer stepping once a second, for two hours [V24]. Detection fell from about 85% to about 75% in 30 minutes. [V11] [inference]

Review becomes ritual

A reviewer who must gate everything clicks through: "a rubber stamp with an audit trail" [V12] [inference]

A critic invents faults

Where models started at $\geq 75\%$ task accuracy, "The critic, primed to find errors, invented them." [V14] [inference]

So tell us when a result is wrong. "the human's job is to adjudicate a raised alarm" [V13]

[V11] [V12] [V13] [V24] Human on the Loop · [V14] Generative Actor Critic

This is the first system the course asks you to look at rather than trust, and it is the one grading you. So here is how it fails. Bainbridge named the general problem in nineteen eighty three. The better the automation, the worse the monitor's job becomes. [S8]

Mackworth measured it. His observers watched a pointer step once a second for two hours, and had to catch its rare double jumps. Detection fell from about eighty five percent to about seventy five within thirty minutes. [V24] [V11]

Second. Make a human sign off everything and you train them to click through. That is a rubber stamp with an audit trail. [V12] Third, and this one bites good work. On the tasks where the models started at seventy five percent accuracy or better, a critic primed to find errors invented them. [V14]

The repair is structural. Report a wrong result to the course staff. A human adjudicates a raised alarm far better than they spot one. [V13]

28 28. What this course is betting

On screen:

Part III · Honesty

The bet: unlimited retries, held by two counterweights — a checker naming defects, and an unaided exam core. [speculative] Three published results argue against it. [inference]

Trial and error [S6]

Repeated submission instead of improving the program. Limits and regression penalties cut its rate by up to 85%. PB158 imposes neither. [speculative]

Rapid feedback [S18]

Some students aim at the tests, not at improving the program.

Guidance [S22]

A learner with low prior knowledge learns less under minimal than under strong guidance.

Edwards' counterweight was student-written tests [S17]. Ours are the two above. [speculative]

[S6] Lawrence · [S18] Luxton-Reilly · [S22] Kirschner · [S17] Edwards

Here is the bet, and it invites attack. The course keeps unlimited retries. Two counterweights carry the risk. The first is a checker that names defects and never remedies. The second is an exam core that every one of you sits unaided. [speculative]

Now the evidence against it. Lawrence and colleagues measured trial and error, which is repeated submission instead of improving the program. Submission limits and regression penalties cut its rate by up to eighty five percent. Students also prefer the limits. [S6] This course imposes neither. [speculative]

Luxton-Reilly and colleagues name the behaviour. With rapid feedback, some students aim at passing the test cases rather than improving the program. [S18] Kirschner, Sweller and Clark compared minimal guidance with strong guidance. Under minimal guidance a learner with low prior knowledge learns less of the material than under strong guidance. [S22]

Their finding is about learners with low prior knowledge of the material in hand. Nobody here used a tracer before. Reading their finding as this course's starting point is an interpretation, not their claim. [speculative] Edwards built unlimited submission in two thousand three with a counterweight of his own. He assessed a student's program only by the tests the student had written. [S17]

The course keeps the unlimited half and substitutes the two counterweights above. If the bet is wrong, it is wrong here. [speculative]

29 29. Where to keep learning

On screen:

Part IV · Reading

1

Julia Evans

javns.ca — why strace fails in Docker. [S12]

2

Raymond Chen

The Old New Thing — Windows history. [S14]

3

Brendan Gregg

Flame graphs, eBPF, this course's overhead numbers. [S13]

4

Oxide and Friends

Bryan Cantrill co-created DTrace, one of the two tracing traditions here. [S16]

Pick one. The handout carries the full annotated list with URLs.

[S12] jvns.ca · [S13] brendangregg.com · [S14] devblogs.microsoft.com/oldnewthing ·
[S16] oxide.computer/podcasts/oxide-and-friends

One minute on reading, and the list is four names. Julia Evans writes about tracing, and about debugging tools. Her piece on why strace does not work inside Docker is exactly the failure you will hit in the first lab. [S12]

Raymond Chen has written The Old New Thing since two thousand three. It is the best single source on why Windows is shaped the way it is. [S14] Brendan Gregg is where this course's tracing overhead numbers come from. [S13] Oxide and Friends is a podcast by Bryan Cantrill, who co-created DTrace. [S16]

Pick one and actually follow it. The full annotated list is in the handout, so do not photograph this slide. [speculative]

30 30. What must work on your Windows machine

On screen:

Part V · Windows

Visual Studio Community

cl.exe, the Windows SDK and PDB symbols. Tick Desktop development with C++.

VS Code, C/C++ extension

The editor, on both platforms.

Sysinternals Suite

Process Monitor and Process Explorer.

NtTrace

Native API tracing, the strace counterpart on Windows.

WinDbg

Crash dumps, for topic 7.

WSL2

How you reach the Linux side from Windows.

Python and uv

A Python package and project manager, written in Rust. [S21]

The course stays Microsoft native. WinDbg reads MSVC symbols. [speculative]

The Windows list. Visual Studio Community first. Tick the workload called Desktop development with C plus plus. That workload carries the compiler. Community is free for classroom use. Build Tools is the wrong package, because it needs a Visual Studio licence.

VS Code with the C and C plus plus extension. The Sysinternals Suite for Process Monitor and Process Explorer. NtTrace, the strace counterpart. WinDbg for topic seven. WSL2, because that is how you reach the Linux side.

And Python with uv, a very fast package manager written in Rust. Use it instead of pip. [S21] One decision worth stating. This is the Microsoft native toolchain by choice, because WinDbg wants MSVC symbols. [speculative]

31 31. What must work on your Linux machine

On screen:

Part V · Linux

VS Code

From code.visualstudio.com, the same editor as the Windows side.

gcc, make, libc headers

Your distribution's build-essential equivalent.

strace and gdb

Your distribution's own packages.

Python and uv

From docs.astral.sh/uv. [S21]

*The course **dev container** carries all of this. A dev container is "a running container with a well-defined tool/runtime stack and its prerequisites" [S10]*

Faculty machines exist for both platforms, so you are not blocked if your laptop refuses. [speculative]

[S10] Development Containers — containers.dev. <https://containers.dev/> · [S21] uv — Astral. <https://docs.astral.sh/uv/>

The Linux list is shorter. VS Code, the same editor. Your distribution's build essential equivalent for gcc, make and the C library headers. strace and gdb from your own package manager. Python and uv again. [S21]

The easy route is the course dev container, which carries all of it preinstalled. A dev container is a running container with a defined tool stack and its prerequisites. A file in the repository describes it, so everyone gets the same environment. If you are on Windows, the dev container is how you reach Linux, through WSL2. [S10]

And if your own laptop refuses any of this, the faculty provides machines for both platforms. Do not spend a week fighting your own hardware. Tell the course staff instead. [speculative]

32 32. Two things that cost an evening

On screen:

Part V · Warnings

1

Visual Studio Community is large

Microsoft's figure is 20 to 50 GB for a typical installation. [S19]

2

cl.exe needs its own shell

"Even a simple Visual Studio installation can set twenty or more environment variables" [S20]

Use the Developer Command Prompt. An ordinary shell says "not recognised", which reads as a broken install rather than a missing environment. [inference]

[S19] Visual Studio 2026 system requirements · [S20] MSVC from the command line — Microsoft Learn. <https://learn.microsoft.com/en-us/visualstudio/releases/2026/vs-system-requirements>

Two warnings, because both cost people an evening every year. First, Visual Studio Community is not a small download. Microsoft's own figure is twenty to fifty gigabytes for a typical installation, and the full range goes up to two hundred and ten. Check your free space first. [S19]

Install it before the first lab rather than tonight, because the first assignment does not need it. Second, `cl dot exe` does not work from an ordinary PowerShell. Microsoft says a simple installation can set twenty or more environment variables, which is why they ship a Developer Command Prompt. [S20]

In an ordinary shell the error just says not recognised, and every year somebody reinstalls Visual Studio Community because of it. [inference]

33 33. Summary — what follows from this

On screen:

Lecture 1 · Close

1

The course never grades recall

The course grades looking, not reciting [T1], and the exam core measures what you can do unaided. [speculative]

2

Every topic goes downward

Your code down to the operating system [T2].

3

Submit the attempt you doubt

The review lectures are built from your failures. [speculative]

4

Check us

The grader has named failure modes [V11] [V12]. A human adjudicates the alarm you raise. [V13]

Now the second lecture: three lines of Python whose output disappears.

[V11] [V12] [V13] Human on the Loop

The two takeaways again, and then two things that follow from them. First, this course grades looking rather than reciting. It never grades recall. The exam core measures what you can do unaided, and the homework is a loop you iterate against. [speculative]

Second, the course works downward, from the code you write to the operating system under it. That is why topic one is a single file write rather than a chapter on kernel architecture.

Then submit the attempt you are unsure about. Your failed attempts are what the review lectures are made of, and a repository of clean passing commits teaches nothing. [speculative]

And check the grading arrangement. It has named failure modes, and part three showed three of them. Raising one is your job as much as the staff's. [V11] [V12] [V13] That is the orientation. Short break, then the actual subject, with three lines of Python whose output vanishes.

34 34. References

On screen:

- **[S1]** *The misunderstood limits of folk science: an illusion of explanatory depth* — Rozenblit & Keil, *Cognitive Science* 26 (2002). <https://cogdevlab.yale.edu/sites/default/files/files/rozenblit%20&%20keil%20%202002.pdf>
- **[S2]** *Classroom demonstrations: learning tools or entertainment?* — Crouch, Fagen, Callan & Mazur, *Am. J. Phys.* 72 (2004). <https://works.swarthmore.edu/fac-physics/203/>
- **[S3]** *Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation* — Neville-Neil and colleagues, *USENIX login* (2017). https://www.usenix.org/system/files/login/articles/login_spring17_03_neville-neil.pdf
- **[S4]** *On teachers working with large-language-model grading in classrooms* — Tian and colleagues (2025), *arXiv:2506.07955*. Recorded title contradicted on re-reading, and no exact title confirmed in this build. <https://arxiv.org/abs/2506.07955>
- **[S5]** *Lessons Learned from 30 Years of MINIX* — Tanenbaum, *CACM* (2016). <https://www.cs.vu.nl/~ast/Publications/Papers/cacm-2016.pdf>
- **[S6]** *On submission limits and regression penalties* — Lawrence, Foss & Urazova, *ACM TOCE* (2023). <https://dl.acm.org/doi/10.1145/3591210>
- **[S7]** *The Keller Plan and the Personalized System of Instruction* — Eyre, *Behav. Analyst Today* 8(3) (2007). <https://files.eric.ed.gov/fulltext/EJ800986.pdf>
- **[S8]** *Ironies of Automation* — Bainbridge, *Automatica* 19(6) (1983). <https://gwern.net/doc/sociology/technology/1983-bainbridge.pdf>

- **[S9]** MU Study and Examination Regulations — Masaryk University, official notice board, current edition. <https://www.muni.cz/en/about-us/official-notice-board/mu-study-and-examination-regulations>
- **[S10]** Development Containers — containers.dev. <https://containers.dev/>
- **[S11]** Create a Dev Container — Visual Studio Code documentation. <https://code.visualstudio.com/docs/devcontainers/create-dev-container>
- **[S12]** "Why strace doesn't work in Docker" — Julia Evans, jvns.ca (2020). <https://jvns.ca/blog/2020/04/29/why-strace-doesnt-work-in-docker/>
- **[S13]** brendangregg.com — Brendan Gregg. <https://www.brendangregg.com/>
- **[S14]** The Old New Thing — Raymond Chen, Microsoft Developer Blogs. <https://devblogs.microsoft.com/oldnewthing/>
- **[S15]** LWN.net — weekly Linux kernel reporting. <https://lwn.net/>
- **[S16]** Oxide and Friends — Bryan Cantrill and Adam Leventhal. <https://oxide.computer/podcasts/oxide-and-friends>
- **[S17]** Web-CAT project paper, pos27-Edwards.pdf — Stephen H. Edwards, Virginia Tech. <https://web-cat.org/publications/pos27-Edwards.pdf>
- **[S18]** On rapid feedback and student submission behaviour — Luxton-Reilly, Tempero & Arachchilage, ACE '23. <https://valerio-terragni.github.io/assets/pdf/luxton-ace-2023.pdf>
- **[S19]** Visual Studio 2026 system requirements — Microsoft Learn. <https://learn.microsoft.com/en-us/visualstudio/releases/2026/vs-system-requirements>
- **[S20]** Using the MSVC toolset from the command line — Microsoft Learn. <https://learn.microsoft.com/en-us/cpp/build/building-on-the-command-line>
- **[S21]** uv — a Python package and project manager — Astral. <https://docs.astral.sh/uv/>
- **[S22]** Why Minimal Guidance During Instruction Does Not Work — Kirschner, Sweller & Clark, Educational Psychologist 41(2) (2006). https://itgs.ict.usc.edu/papers/Constructivism_KirschnerEtAl_EP_06.pdf
- **[V1] [V2] [V3]** Observability vs Monitoring — author's working notes.
- **[V4] [V5] [V6]** Where the Abstraction Leaks — author's working notes.
- **[V7]** Sockets and Network I/O — author's working notes.
- **[V8]** Windows and Linux place the stable syscall boundary at different layers — author's working notes.
- **[V9] [V23]** Observing a Running System — author's working notes.
- **[V10] [V11] [V12] [V13] [V24]** Human on the Loop — author's working notes.

- **[V14] [V15]** *Generative Actor Critic as an Agentic Orchestration Pattern* — author's working notes.
- **[V16]** *Goodhart's Law* — author's working notes.
- **[V17]** *Virtual Memory and the Page Cache* — author's working notes.
- **[V18]** *Memory Protection* — author's working notes.
- **[V19]** *Processes, Threads and Scheduling* — author's working notes.
- **[V20]** *Crash Dump Analysis on Windows and Linux* — author's working notes.
- **[V21] [V26] [V27]** *AI Agents as Debugging Operators* — author's working notes.
- **[V22] [V25]** *Every Tracing Mechanism Trades Observation Cost for a Blind Spot* — author's working notes.

The handout PDF carries the full reference list, and a one-page summary of every [V] source with its own external references.

The full source list. Every claim in this lecture points at one of these. The handout uses the same numbering, and its appendix summarises each of the working notes. Check any claim in this lecture there.